

DOCUMENTO DE REQUISITOS DEL PRODUCTO (PRD)

Proyecto: E-Commerce MVP – Entrega Final de Portafolio (SQLite3 + Procfile/Build Script)

1. Control de Versiones y Metadatos

- **Autor:** Cristian Lopez
- **Estado:** Listo para Implementación
- **Entorno de Base de Datos:** SQLite3 (Almacenamiento local)
- **Estrategia de Despliegue:** Render Web Service manual mediante `build.sh` y `Procfile`

⚠

NOTA CRÍTICA SOBRE SQLITE3 EN RENDER:
Los servicios web en la capa gratuita de Render poseen un sistema de archivos efímero. Esto significa que cada vez que el contenedor se reinicie por inactividad o un nuevo despliegue, el archivo `db.sqlite3` se restablecerá. Para mitigar esto en un entorno de portafolio, se integra una precarga automatizada de catálogo (vía fixtures) dentro del script de construcción.

2. Propósito y Objetivos del Producto

El propósito de este desarrollo es consolidar un MVP funcional de comercio electrónico adaptado para una presentación ágil de portafolio. El diseño del despliegue omite deliberadamente configuraciones complejas de orquestación de infraestructura (como los planos de control YAML), optando en su lugar por un flujo imperativo basado en un script de compilación nativo de Linux y un archivo de procesos explícito.

3. Alcance del MVP y Requisitos Funcionales

3.1 Autenticación y Matriz de Accesos

Rol de Usuario	Vistas y Acciones Permitidas	Comportamiento del Sistema
Invitado (Anónimo)	Navegación general por el catálogo de productos y visualización de detalles.	Si intenta agregar productos al carrito, es redirigido al flujo de Login.
Cliente (Autenticado)	Operación completa del carrito (agregar, quitar, cambiar cantidad) y confirmación de pedidos.	Acceso restringido a su propio historial de compras y carrito persistido por sesión.
Administrador (Staff)	Acceso total al módulo CRUD de administración de productos y catálogo comercial.	Protección mediante decoradores <code>@staff_member_required</code> en backend.

3.2 Carrito de Compras y Lógica de Negocio

El núcleo transaccional calcula los montos de forma dinámica asegurando consistencia matemática antes de la persistencia:

- **Subtotal por Producto:** Se computa mediante la relación lineal: $Subtotal = Precio\ Unitario \times Cantidad$
- **Total del Carrito:** Sumatoria algebraica explícita: $Total = \Sigma(Subtotales)$
- **Validación de Inventario:** No se permiten adiciones de cantidades que excedan el *stock* actual del registro en la tabla de productos.

3.3 Confirmación de Pedidos

Al accionar "Confirmar Compra", el sistema ejecuta de manera atómica:

1. Generación de una instancia en el modelo Orden vinculada al User autenticado.
2. Migración de los ítems activos del carrito hacia registros rígidos en el modelo OrdenItem.
3. Deducción del inventario remanente y vaciado absoluto de la estructura del carrito.

4. Arquitectura de Configuración y Despliegue en Render

A diferencia de configuraciones declarativas complejas, el despliegue se estructurará mediante scripts directos en la raíz del repositorio de GitHub.

4.1 Script de Construcción: build.sh

Este archivo automatiza la preparación del entorno Linux en Render. Debe crearse en la raíz con codificación de saltos de línea LF.

```
#!/usr/bin/env bash
# Salir inmediatamente si un comando falla
set -o errexit

echo "--- Iniciando proceso de instalación de dependencias ---"
pip install -r requirements.txt

echo "--- Compilando archivos estáticos de la aplicación ---"
python manage.py collectstatic --noinput

echo "--- Aplicando migraciones estructurales a SQLite3 ---"
python manage.py migrate

echo "--- Cargando catálogo base desde archivos de fixture ---"
if [ -f "productos_iniciales.json" ]; then
    python manage.py loaddata productos_iniciales.json
else
    echo "Aviso: No se encontró el archivo productos_iniciales.json para precarga."
fi

echo "--- Proceso de construcción finalizado con éxito ---"
```

4.2 Archivo de Procesos: Procfile

Determina la instrucción explícita del servidor de aplicaciones de producción que mantendrá el servicio web escuchando peticiones HTTP.

```
web: gunicorn tu_proyecto.wsgi:application
```

4.3 Archivo de Dependencias: requirements.txt

```
django
gunicorn
whitenoise
```

5. Configuración en el Dashboard de Render

Para desplegar la aplicación manualmente sin usar archivos YAML estructurales, sigue estos pasos en la interfaz gráfica de Render:

1. Haz clic en **New +** y selecciona **Web Service**.
2. Conecta tu cuenta de GitHub y selecciona el repositorio correspondiente al e-commerce.
3. En la sección de configuraciones del entorno, define los siguientes parámetros:
 - **Runtime:** Python
 - **Build Command:** `./build.sh`
 - **Start Command:** `web` (Render autodetectará el archivo Procfile si dejas este campo alineado, o puedes escribir directamente `gunicorn tu_proyecto.wsgi:application`)
4. Despliega la sección **Advanced** y añade las siguientes Variables de Entorno (Environment Variables):
 - `PYTHON_VERSION:` `3.11.5` (o tu versión local estable)
 - `DJANGO_SETTINGS_MODULE:` `tu_proyecto.settings`
 - `SECRET_KEY:` *[Tu clave secreta de producción]*
5. Haz clic en **Create Web Service**.

6. Criterios de Aceptación y Validación Final

- ☒ El archivo `build.sh` tiene otorgados permisos de ejecución previos al push (`git update-index --chmod=+x build.sh`).
- ☒ Los archivos estáticos son servidos correctamente en producción a través de la integración de **WhiteNoise** en el archivo `settings.py`.
- ☒ La plataforma arranca con un catálogo poblado automáticamente gracias al comando `loaddata` embebido en la compilación.